



Tema 17: Deep Learning II

Contenidos

- Series temporales
- Redes neuronales recurrentes
- Redes LSTM
- Procesamiento del lenguaje
- *Embeddings*
- Casos de uso



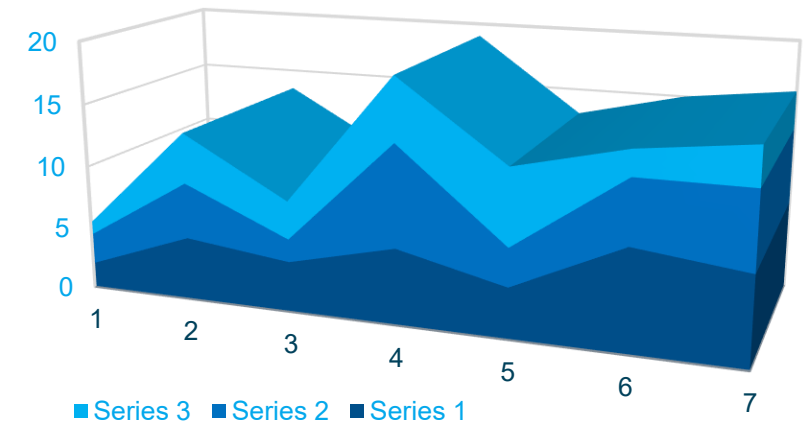
1.Series temporales



Observaciones ordenadas de una o varias variables:

$$(x_0, x_1, x_2, \dots, x_T)$$

- Variables meteorológicas (presión, temperatura, humedad...).
- Texto: caracteres o palabras en un párrafo.
- Audio/voz: amplitud sonora.
- Finanzas: precios de activos.
- Sensores biométricos: ECG, EEG...
- Vídeos: secuencias de imágenes ordenadas.



Variables que muestran dependencia temporal y autocorrelación: los valores en un instante dependen de los valores anteriores.

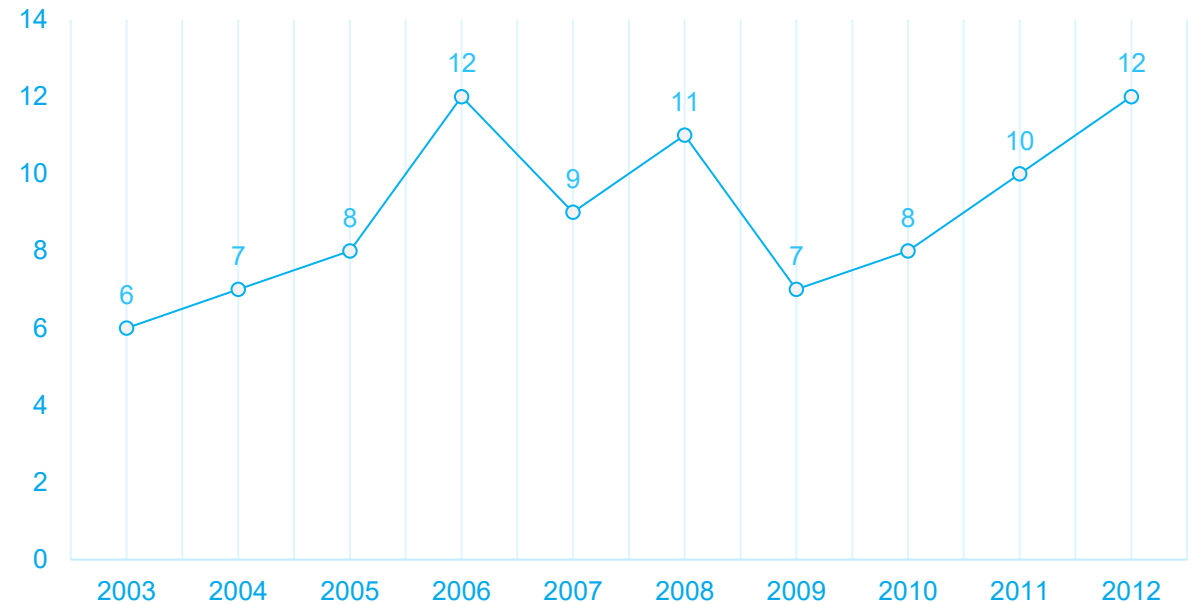


Características de datos ordenados:

- Tendencia.
- Estacionalidad.
- Componente irregular / intermitente.

Modelos clásicos:

- Suavizado exponencial.
- ARIMA.
- Croston.
- Autorregresivos.
- Prophet.



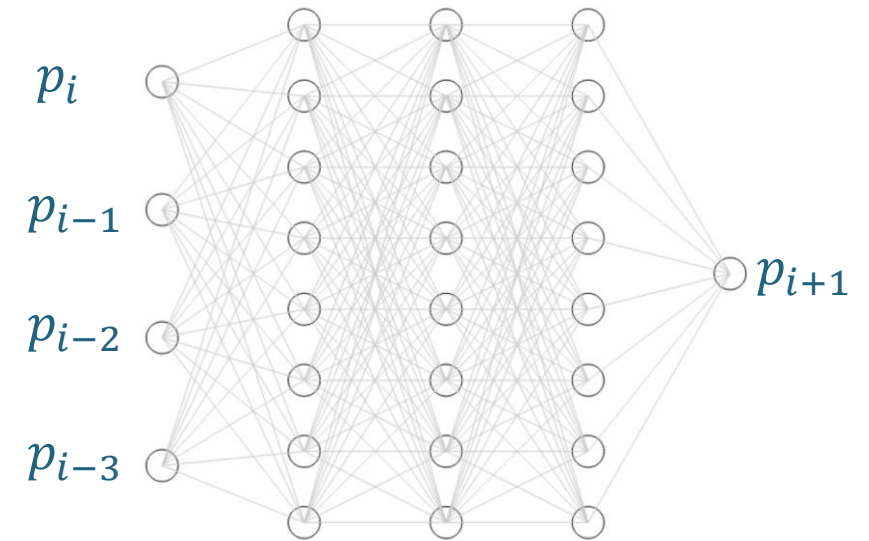
¿Podemos usar MLPs o CNNs?



Ejemplo: serie temporal del precio horario p_i de un activo a lo largo del tiempo con una única variable de entrada:

$$p_0, p_1, p_2, \dots, p_T$$

- Objetivo: predicción de p_i en $T + 1, T + 2, \dots$
- Posible solución: usamos *lags* (en $T - 1, T - 2, \dots$)



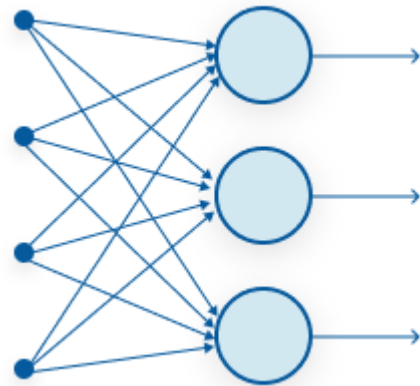
No da buenos resultados

2. Redes neuronales recurrentes

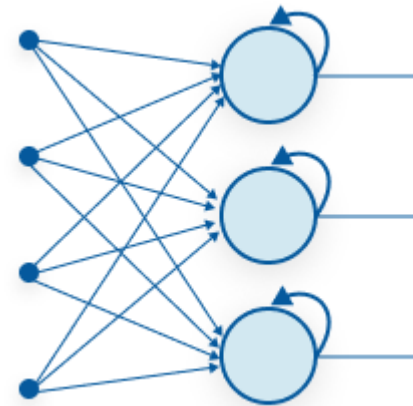


Añadimos conexiones hacia atrás en la red neuronal, conexiones *backwards*.

Red feed forward

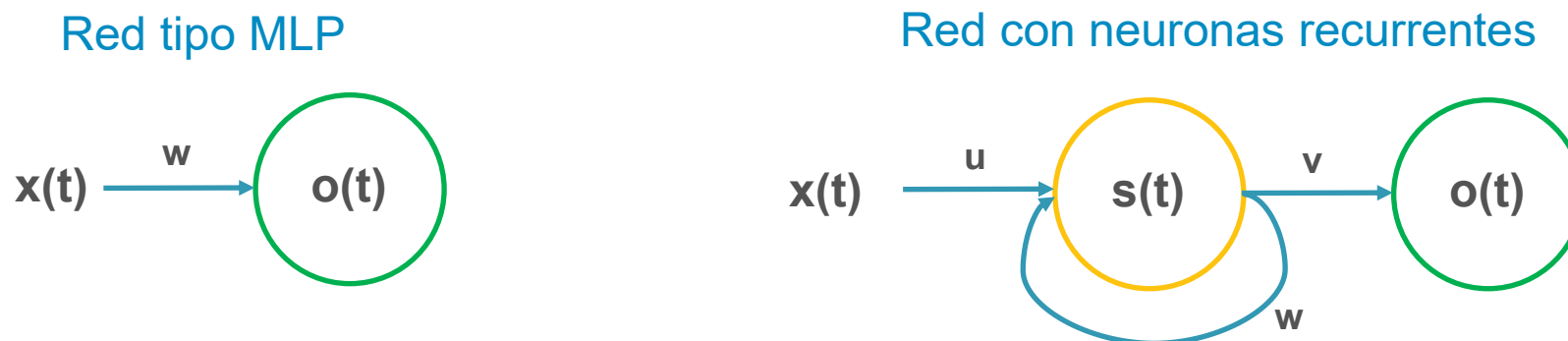


Red con conexiones hacia atrás



- De esta manera, todos los nuevos *outputs* dependen de los anteriores *inputs*: introducimos información temporal.
- Estas conexiones hacia atrás se denominan *ciclos dirigidos* o *delays*.

En lugar de tener *input* $\rightarrow$ *output*, añadimos un estado oculto intermedio:



$$o(t) = g(v \cdot s(t))$$

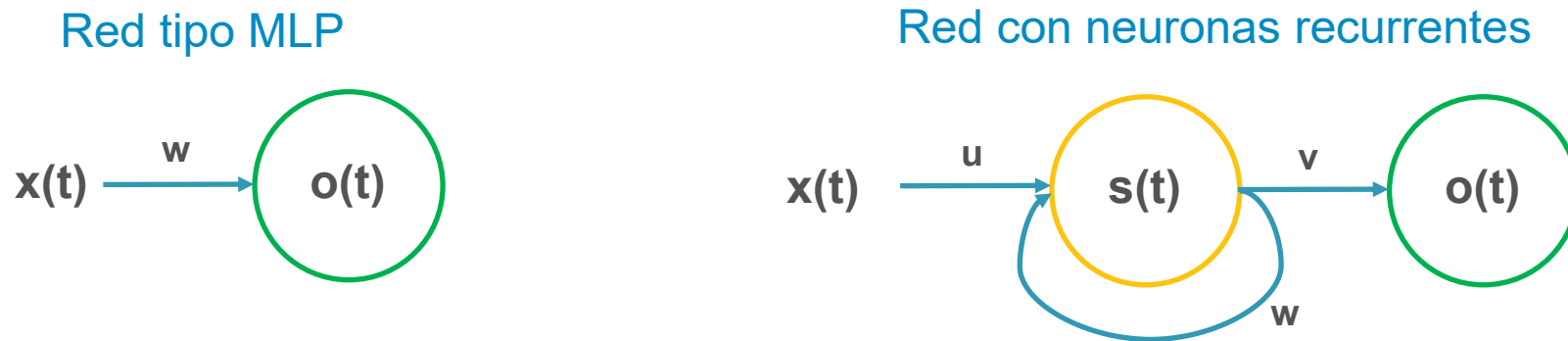
$$s(t) = f(w \cdot s(t-1) + u \cdot x(t))$$

Denominamos estado oculto a $s(t)$.

- $s(t)$ depende del *input* en ese tiempo y el estado oculto del paso anterior.
- El *output* depende del estado oculto únicamente.



En lugar de tener *input* $\rightarrow$ *output*, añadimos un estado oculto intermedio:



$$o(t) = g(v \cdot s(t))$$

$$s(t) = f(w \cdot s(t-1) + u \cdot x(t))$$

Por cada neurona recurrente tenemos:

- Dos funciones de activación (f , g).
- Tres parámetros (u , v , w).



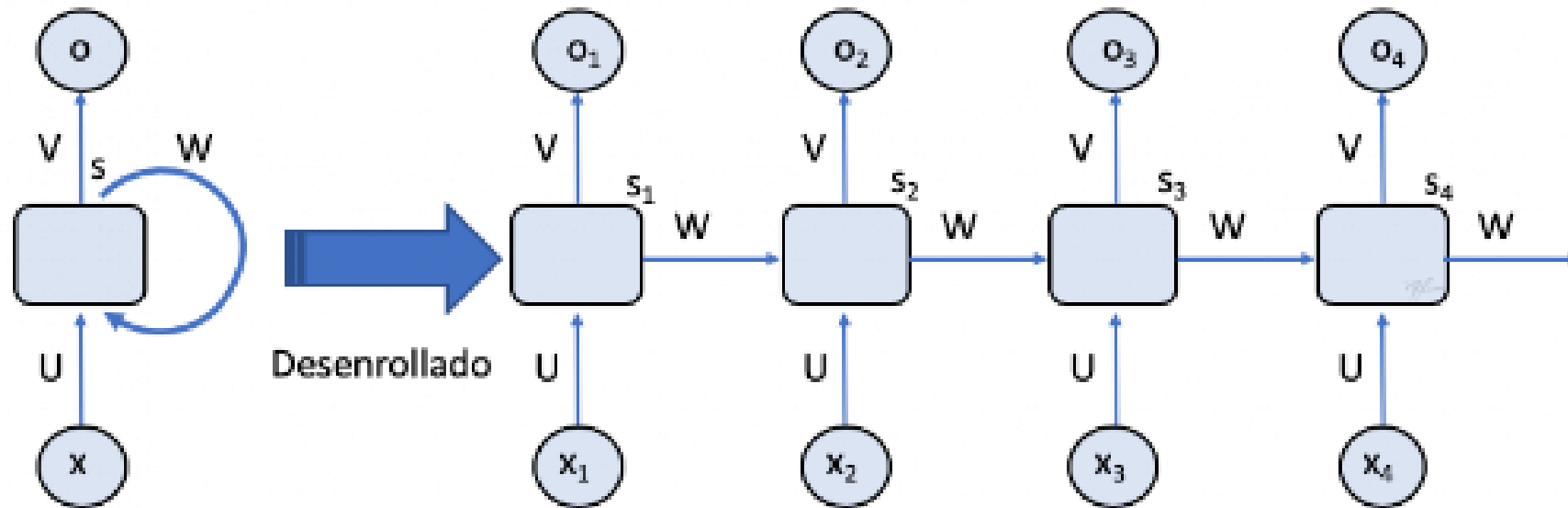
Las redes recurrentes arreglan la mayoría de los puntos débiles de los MLP para procesar series temporales.

	Red <i>feed-forward</i> (MLP)	Red recurrente (RNN)
Flujo de datos	Solo hacia delante	Cíclico en el tiempo
Contexto	Ninguno	Conserva estado histórico s_t
Tamaño de entrada	Fijo	Variable (secuencias)
Generalización a longitudes nuevas	No	Sí
Parámetros	Uno por paso (<i>lags</i>)	Compartidos en el tiempo

La clave está en el almacenamiento de información de pasos anteriores (“**memoria**”) en el estado oculto, s_t .



Visualizamos las RNN como un MLP a través del desdoblamiento



Se entrena como un MLP pero:

- Retropropagación hacia atrás en el tiempo (BPTT).
- Limitamos el número de capas hacia atrás.

	Entrada	Salida	Ejemplos
Many-to-one	$x_1, \dots, x_T$	y único	Análisis de sentimientos, clasificación audio (secuencia $\rightarrow$ etiqueta)
One-to-many	x único	$y_1, \dots, y_T$	Generación texto a partir de etiquetas, descripción de imágenes ...
Many-to-many sincrónico	x_t	y_t	Etiquetado gramatical, subtitulado <i>frame-wise</i>
Many-to-many asincrónico	$x_1, \dots, x_T$	$y_1, \dots, y_{T'}$	Traducción, generación de resúmenes, texto a voz, <i>chatbots</i> ...

Normalmente las RNN se combinan con otras arquitecturas para poder enfrentarse a todas las situaciones descritas en la tabla.



- 🗣️ Procesamiento de Lenguaje Natural (NLP): Modelado de lenguaje, predecir la siguiente palabra en una secuencia, traducción automática, análisis de sentimientos, resumir texto, reconocimiento de entidades...
- 🧠 Series temporales y pronósticos: predicción de precios financieros, pronóstico de demanda (retail, logística, energía...). Análisis de datos biométricos (actividad cerebral, ritmo cardíaco...).
- 🎧 Reconocimiento y síntesis de voz, transcripción de voz a texto y de texto a voz.
- 🎵 Aplicaciones musicales: composición automática, generación de melodías, reconocimiento de música y ritmo...
- 🎥 Visión por computador con secuencias: reconstrucción de acciones en vídeo, seguimiento de objetos.
- 🧬 Otras áreas: generación de texto en imágenes (*captioning*), análisis de ADN y proteínas.

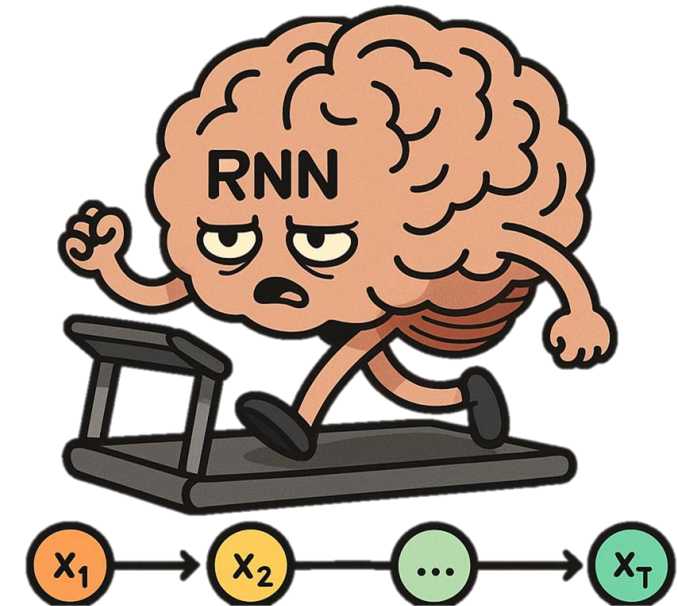
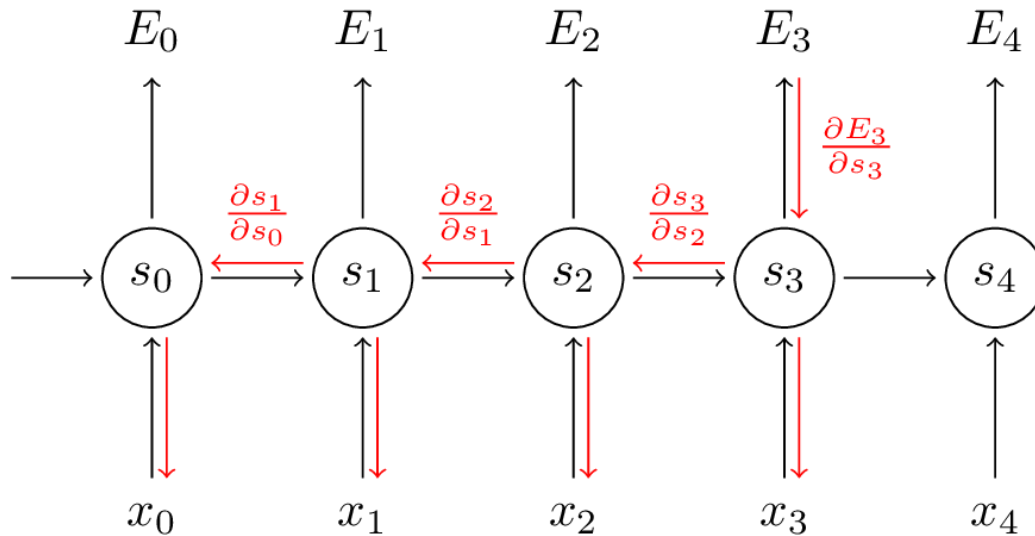


3. Redes LSTM



Dificultades en el entrenamiento y en el rendimiento:

- Problemas del BPTT... explosión y desvanecimiento de gradientes.
- Memoria a corto plazo.

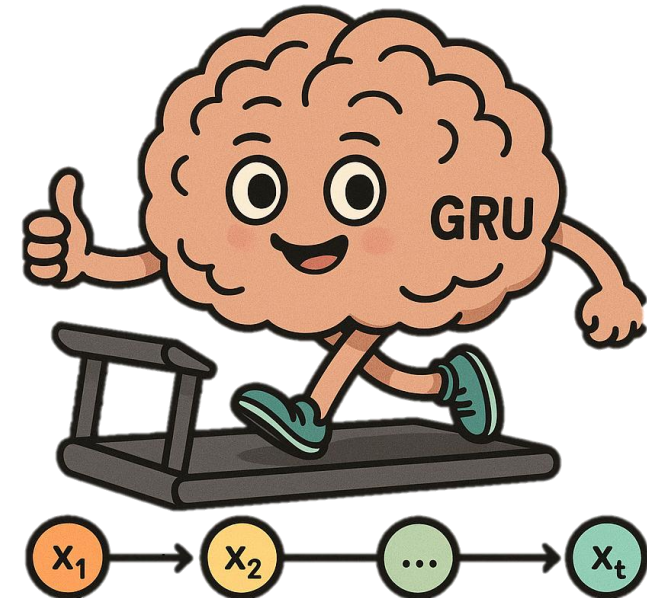


Aplicación de diferentes técnicas:

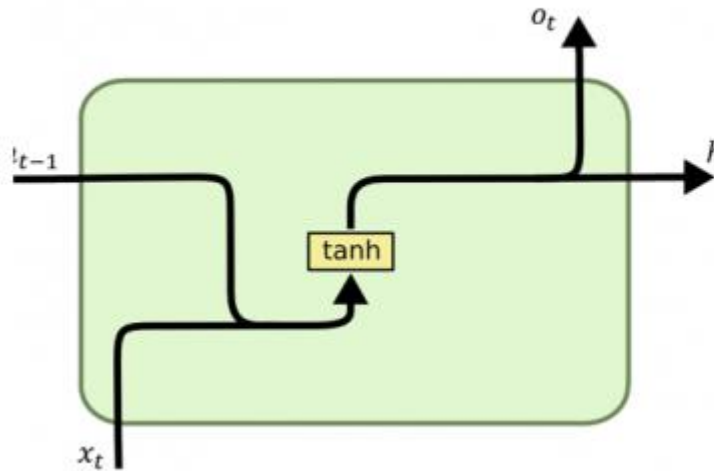
- Limitar los gradientes (*gradient clipping*).
- Normalización por lotes (*batch normalization*).
- Inicialización de pesos (He/Kaiming, Xavier Glorot).

Arquitecturas avanzadas:

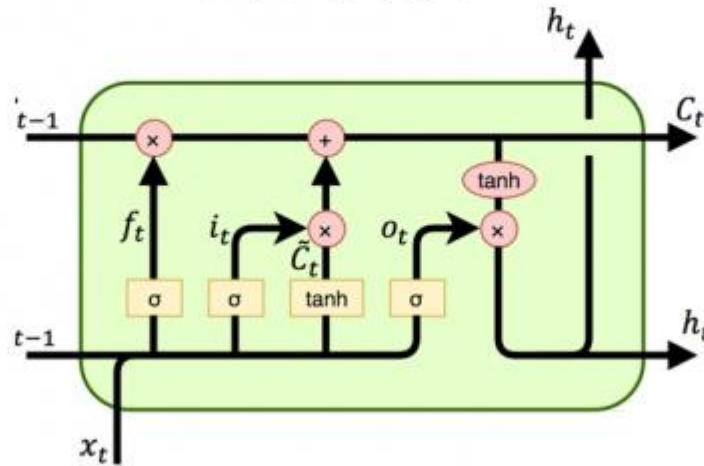
- *Long short term memory* (LSTM).
- *Gated recurrent unit* (GRU).



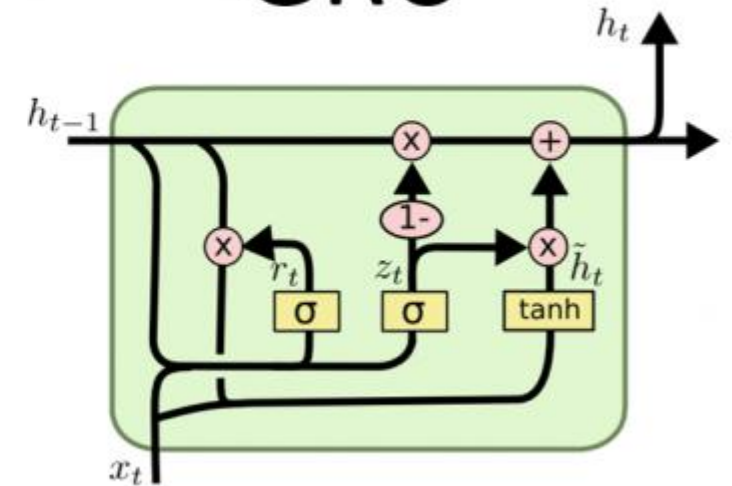
RNN



LSTM

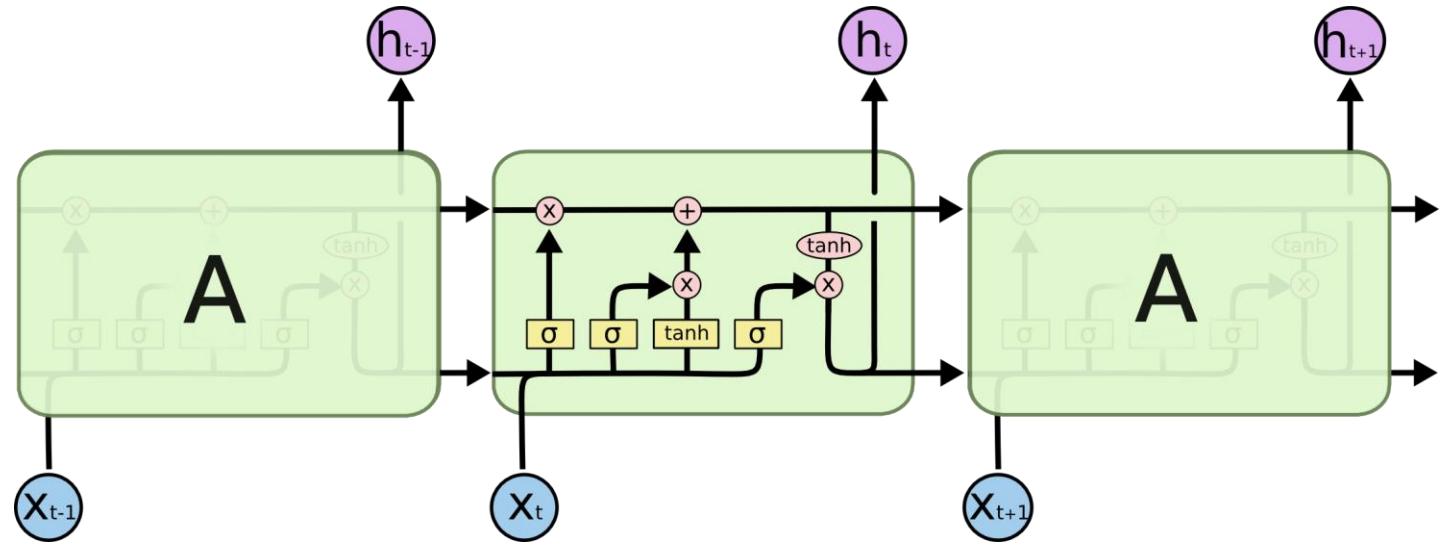


GRU



Algunas propiedades:

-  Memoria a largo plazo.
-  Puertas de control.
-  Estado de celda.
-  Evitan el desvanecimiento del gradiente.
-  Mejor rendimiento que RNN simples en tareas de secuencia con dependencias largas.
-  Más costosas computacionalmente que las RNN básicas.



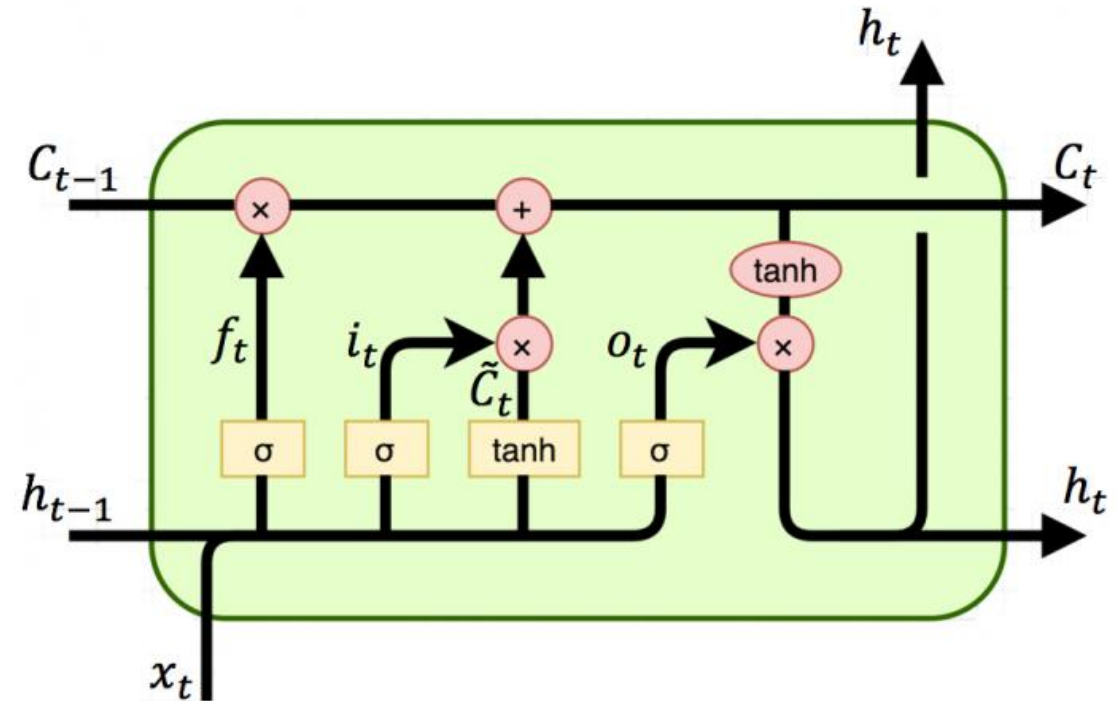
Funcionamiento de una LSTM

Resumen:

x_t : input.

h_t : output.

C_t : estado de la celda.

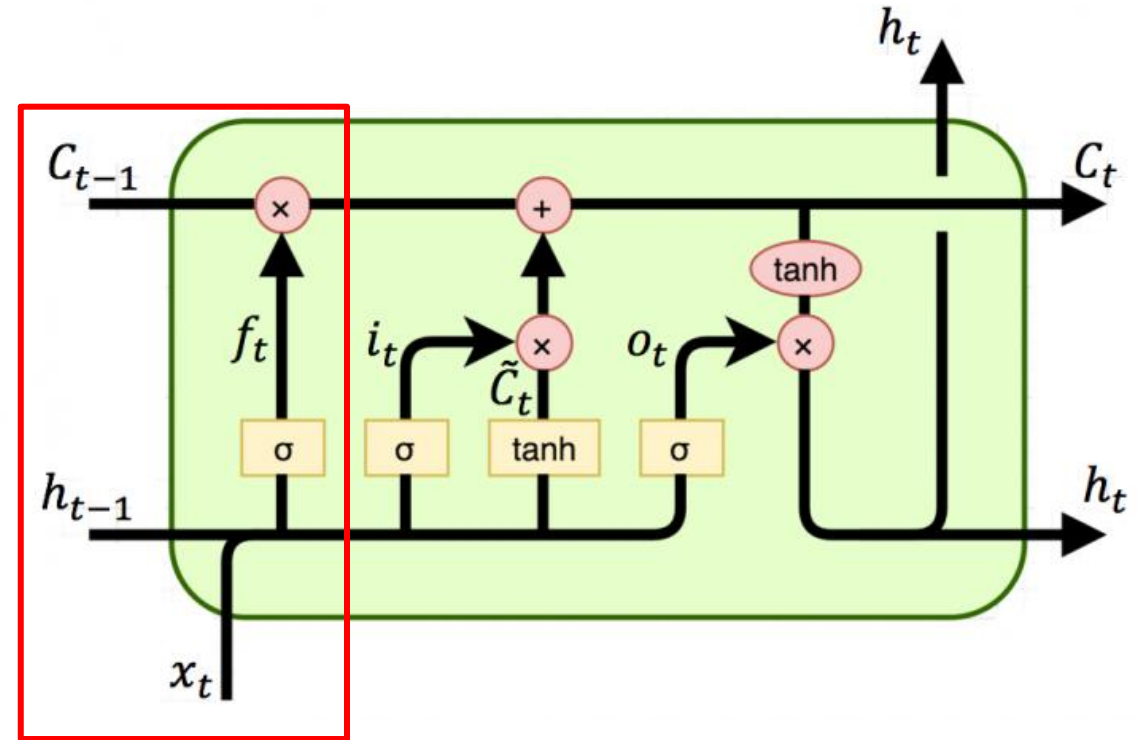


$\sigma \rightarrow 0$ = no deja pasar nada, 1 = máximo efecto.



Puerta de olvido (*forget gate*):

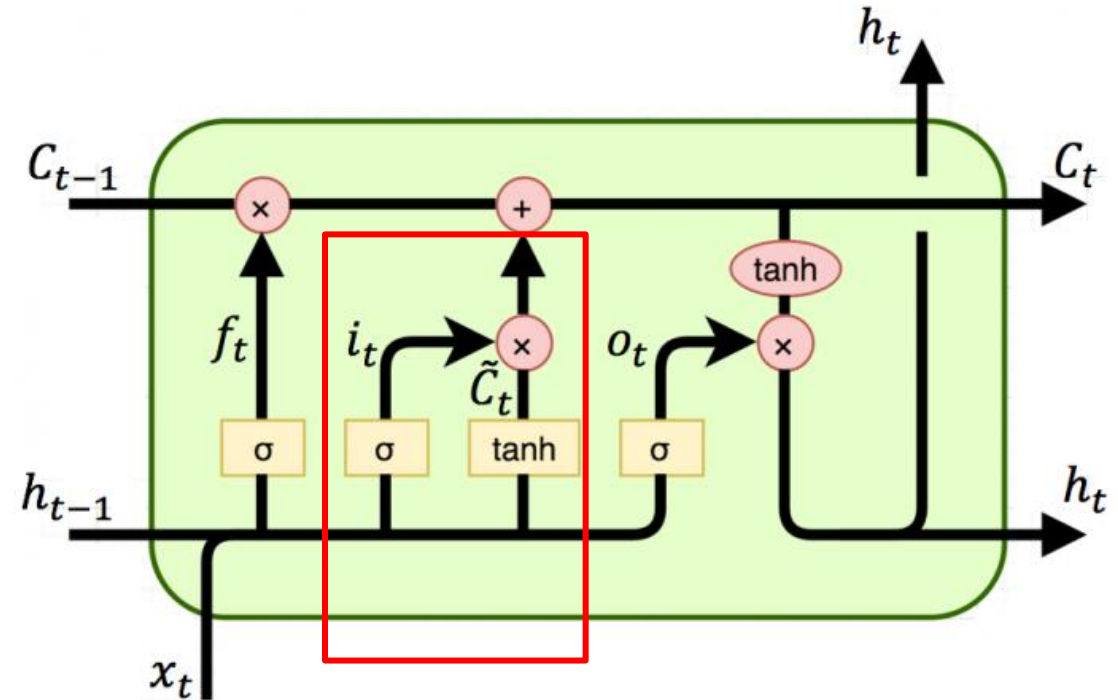
- Calcula el efecto del *input* sobre el estado de la celda histórico.
- Cuánto afecta este *input* en el estado anterior: cuánto debemos olvidar de lo anterior.



¿Qué información antigua ya no necesito?

Puerta de entrada (*input gate*):

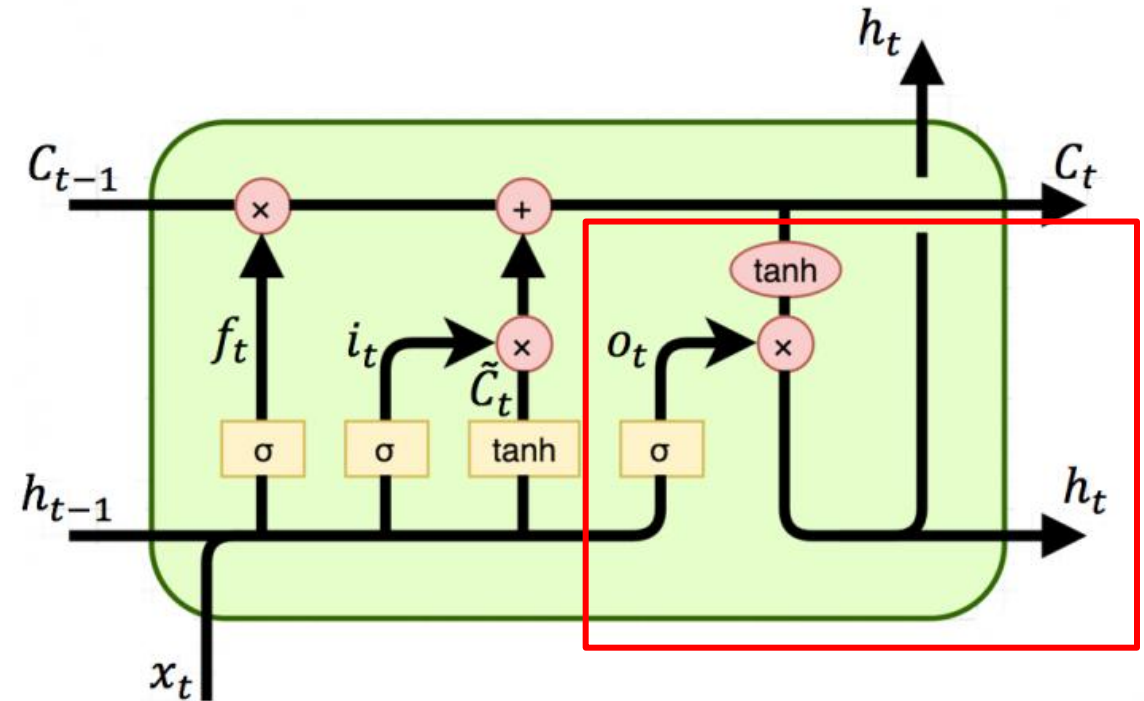
- Calcula el efecto del *input* sobre el estado de la celda presente.
- Cómo se modifica el estado de la celda debido a este nuevo *input*.



¿Qué información nueva es importante guardar?

Puerta de salida (*output gate*):

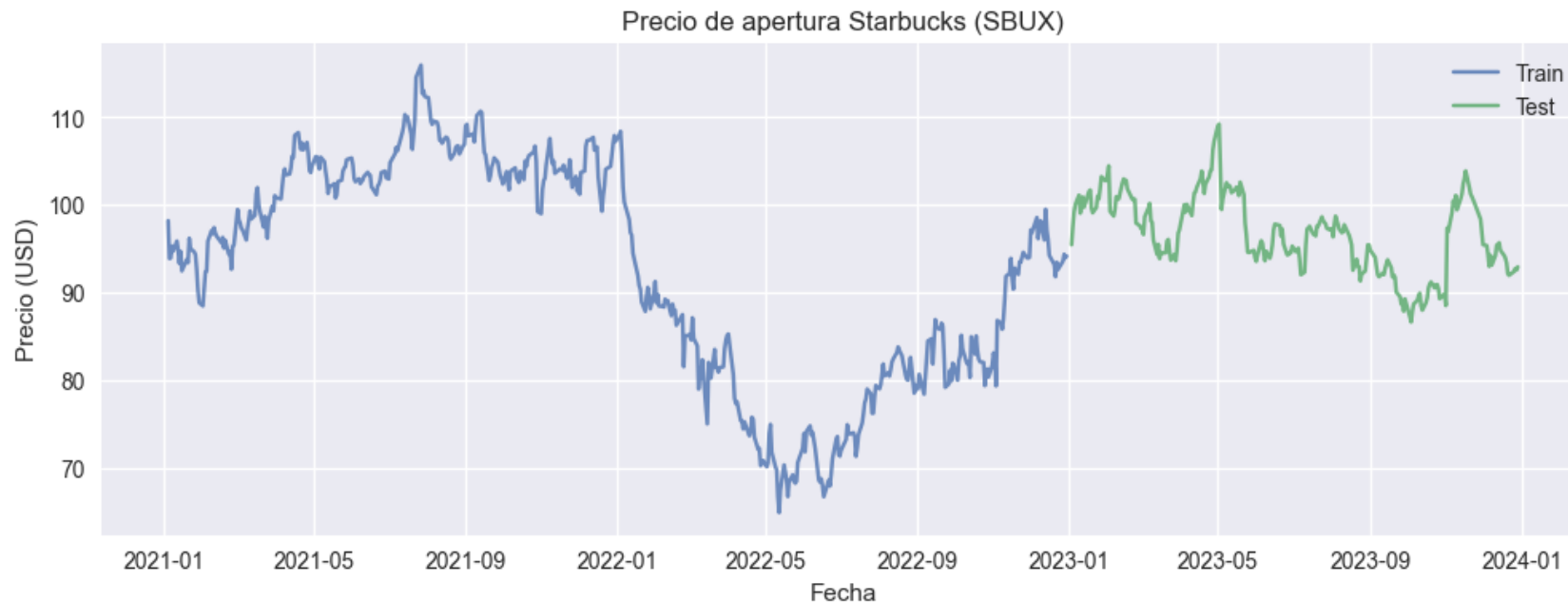
- Calcula el efecto del *input* sobre la salida actual.
- La salida tiene en cuenta tanto el estado de la celda como la entrada actual y el anterior *output*.



¿Qué parte de mi memoria uso ahora para responder?

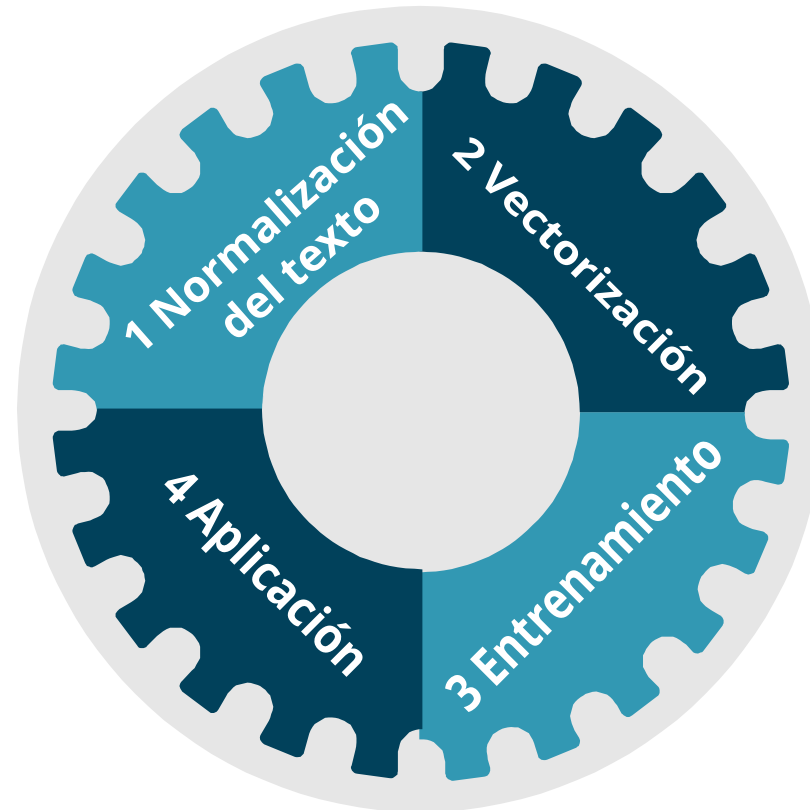
Vamos a entrenar una LSTM para predecir el precio de apertura de las acciones del día siguiente.

- Datos de *yahoo finance* desde Python.
- Entrenaremos una LSTM para, a partir de los últimos datos, predecir el siguiente.



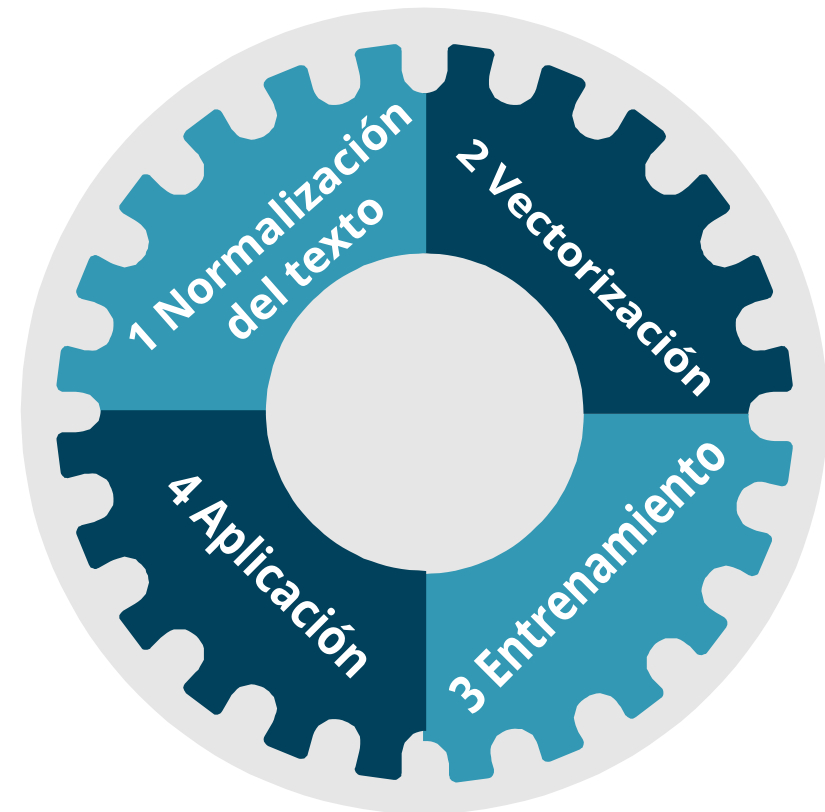
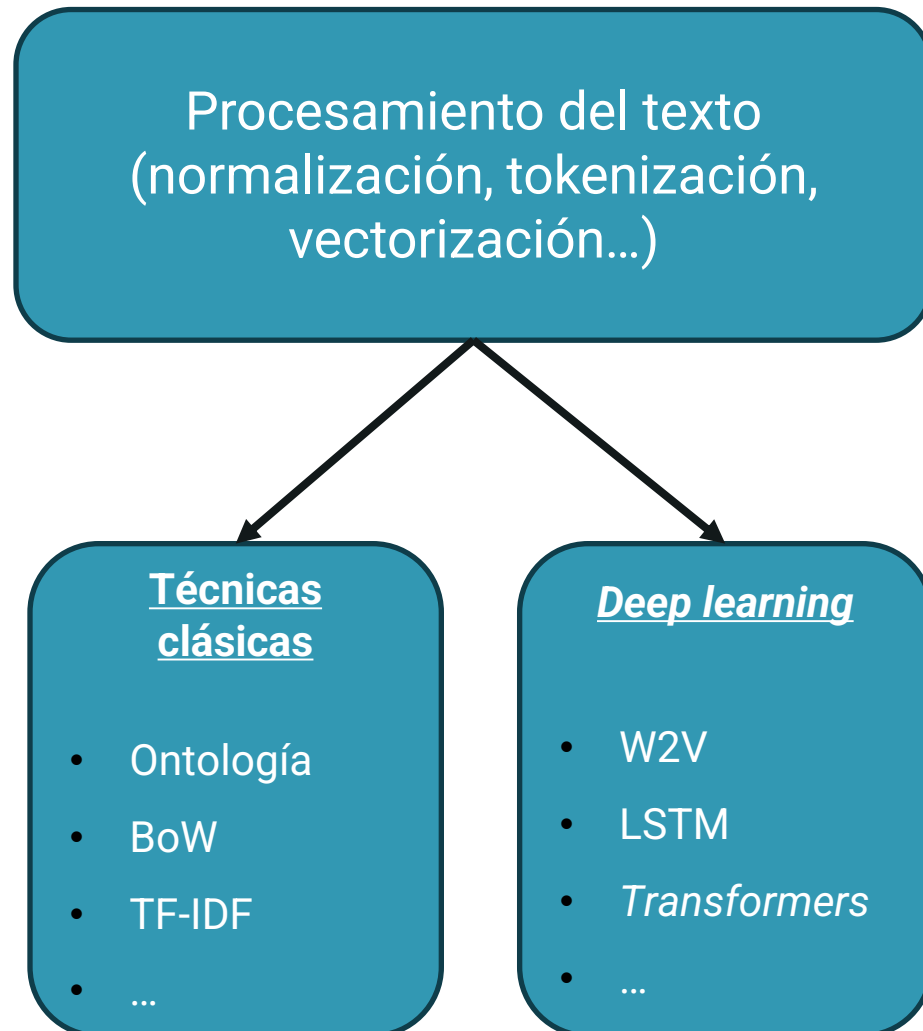
Lenguaje escrito = secuencia ordenada de palabras. Podemos usar una LSTM.

- Clasificación de textos.
- Análisis de sentimiento.
- Asignación preguntas / respuestas.
- Generación de lenguaje.



4. Procesamiento del lenguaje

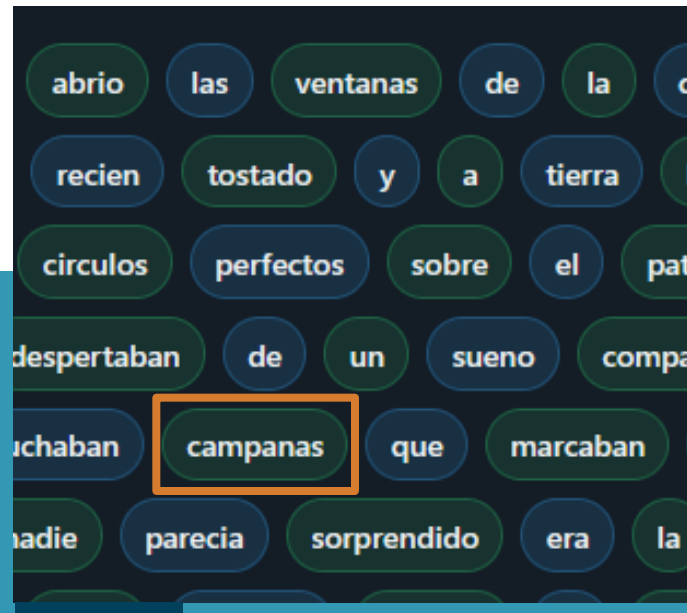




cuando ursula abrio las ventanas de la casa el
aire olia a cafe recien tostado y a tierra humeda
los pajaros tejian circulos perfectos sobre el
patio mientras los vecinos despertaban de un
sueno compartido a lo lejos se escuchaban
campanas que marcaban las horas al reves
pero nadie parecia sorprendido era la senal de
que el rio habia decidido cambiar su cauce
durante la noche los ninos recogian flores que
brillaban como carbones encendidos y las
llevaban a la plaza donde una mujer leia cartas
que siempre hablaban del futuro pasado

Normalización

Eliminamos caracteres especiales,
transformamos a minúsculas...



Tokenización

Dividimos en palabras / sílabas / letras...



Vectorización

Asignamos un vector a cada palabra (*one-hot encoding, embeddings...*).

Se utiliza para dividir y estandarizar el texto en elementos simples:

- Se quitan símbolos.
- Se pasa todo a minúsculas.
- Se quitan o unifican tildes.
- Se remplazan cosas como URLs y citas.

Sin normalizar:

¡Hola! Mi correo es ejemplo@dominio.com y visítame en
https://www.ejemplo.com. Tengo 2 perros. ¿Cómo estás?

Normalizado:

¡hola! mi correo es <EMAIL> y visitame en <URL>.
tengo <NUM> perros. ¿como estas?



- Se asigna un token a palabras, o sub-palabras, o caracteres (hay diferentes estrategias).
- Se asigna un token al “final de frase” y/o los signos de puntuación.
- Se genera el diccionario de **todos los tokens posibles**.
- Dependiendo de la estrategia, se puede generar un índice y/o vector a cada palabra del diccionario.

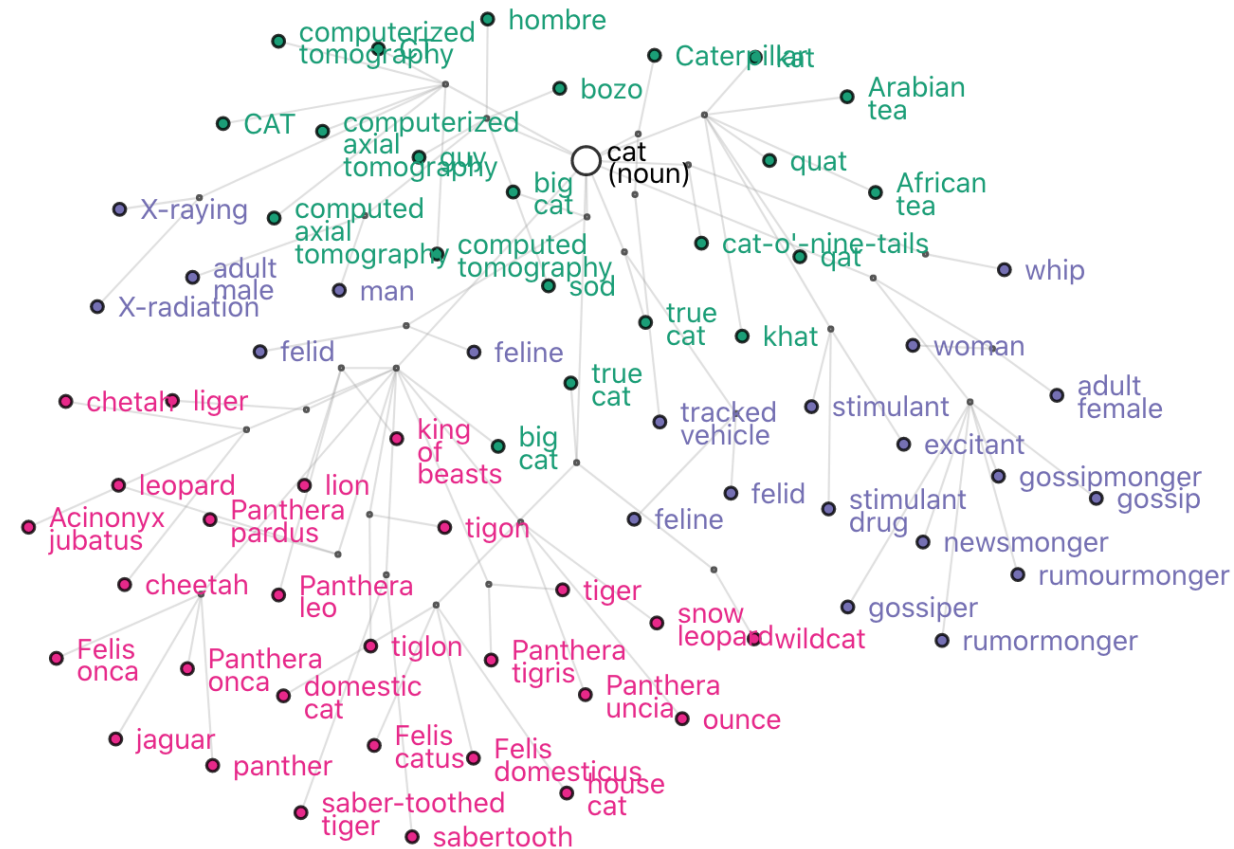
TOKENIZACIÓN



Una aproximación clásica: uso de ontologías predefinidas para estandarizar relaciones, términos, etc.

La más conocida: WordNet.

- Jerarquización: clases (conceptos), instancias (individuos), propiedades (atributos y relaciones).
- Incluye definiciones, sinónimos y antónimos, etc.
- Operaciones: desambiguación, amplitud semántica, búsquedas, cuantificar relaciones, etc.



Gato → es un → animal
Animal → tiene → patas
Gato → caza → ratones



Bag-of-words (BoW)

Modelo sencillo basado en contar palabras e indexarlas.

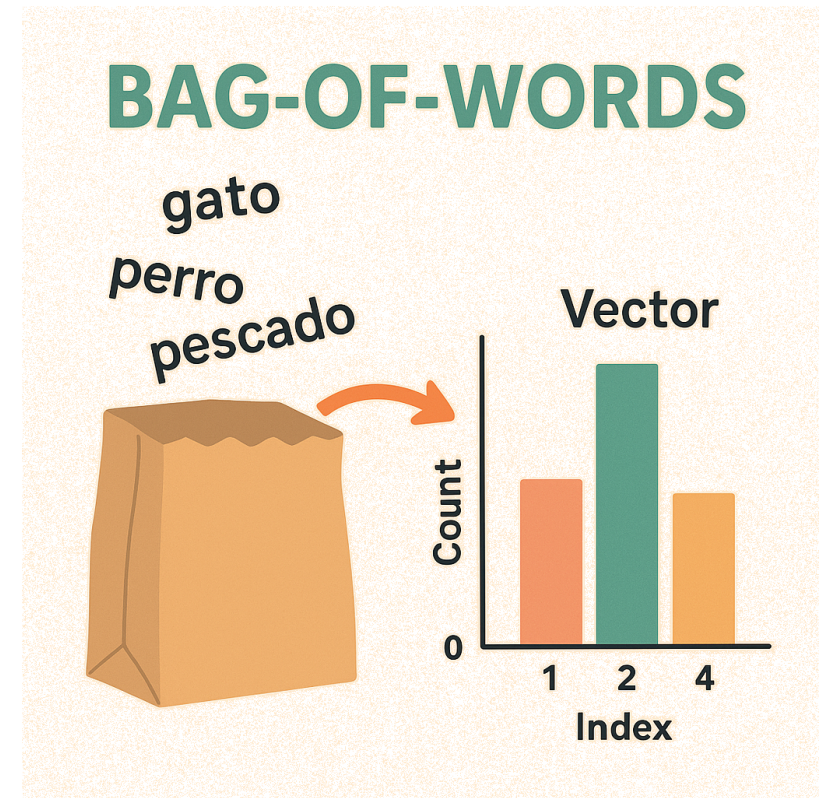
Se pueden realizar estadísticas sencillas, *clustering*, etc.

En un vocabulario de 6 palabras:

- Una palabra: [0, 0, 0, 0, 1, 0]
- Una frase de 3 palabras: [0, 1, 1, 0, 1, 0].

Vocabulario: [el, la, gato, perro, come, pescado]

“El gato come pescado” $\longrightarrow$ **[1, 0, 1, 0, 1, 1]**



Term frequency – inverse document frequency

Se ponderan las palabras para favorecer las más relevantes.

- Documentos d , términos t . Frecuencia del término en un documento:

$$tf(t, d) = \frac{\sum t \in d}{\sum \text{todos los términos} \in d}$$

- IDF: contamos en N documentos en cuántos de ellos aparece el término t ($\{d: t \in d\}$).

$$idf(t) = \log \left(\frac{N}{1 + |\{d: t \in d\}|} \right)$$

Ordenación en base a $TF - IDF = tf(t, d) \times idf(t)$.



5. Embeddings



El vector que representa el token es un vector de ceros, con longitud igual a la longitud del diccionario, y un valor 1 en la posición de su índice.

Por ejemplo:

1. Vocabulario en este diccionario: (perro, casa, árbol, playa).
2. Asignación de índices: {perro: 0, casa: 1, árbol: 2, playa: 3}.
3. $\text{Len}(\text{diccionario}) = 4$, entonces, partimos de $[0,0,0,0]$.

perro: $[1, 0, 0, 0]$

casa: $[0, 1, 0, 0]$

árbol: $[0, 0, 1, 0]$

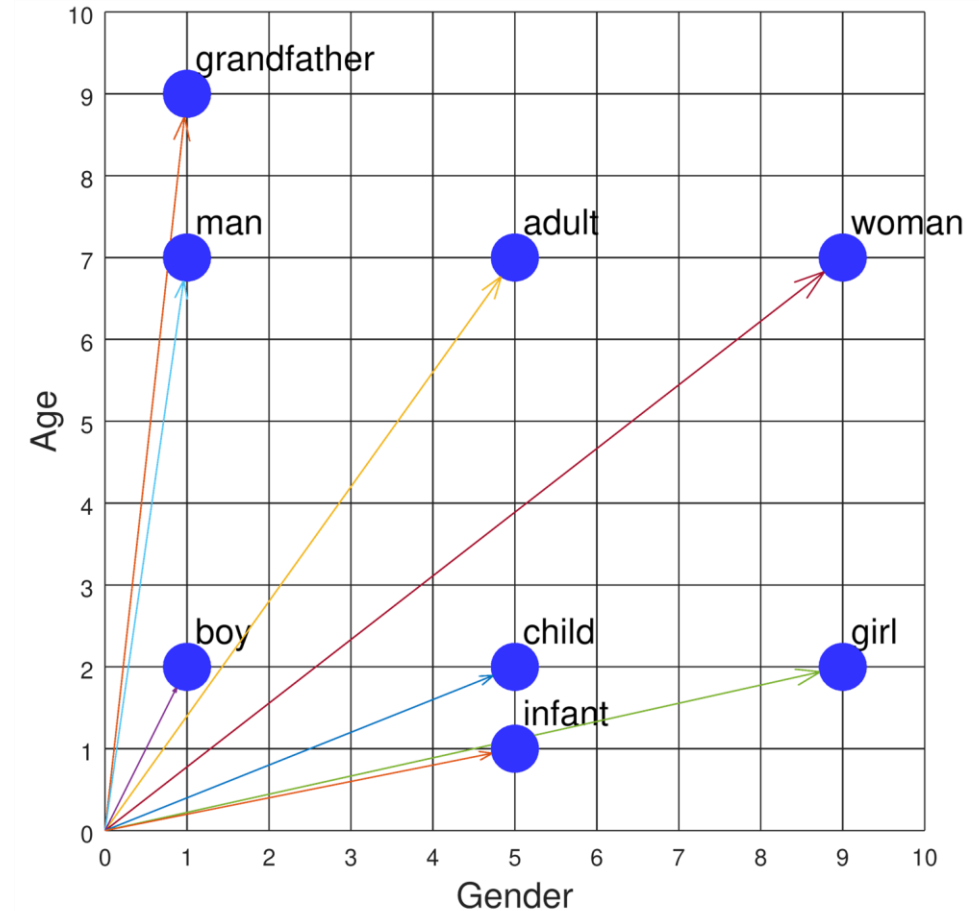
playa: $[0, 0, 0, 1]$

One-hot identifica palabras, pero no entiende nada sobre ellas.



Vectores creados con información semántica. Las ideas principales son:

- Vectores densos, de *floats*, de longitud L mucho menor que el tamaño del diccionario D ($L \ll D$).
- Los vectores codifican información semántica y/o léxica.
- Se pueden entrenar de manera independiente de otros modelos, y utilizar como codificación inicial, como *input* a otros modelos (por ejemplo, a modelos del lenguaje).



Entrenamiento de un modelo de *embeddings*

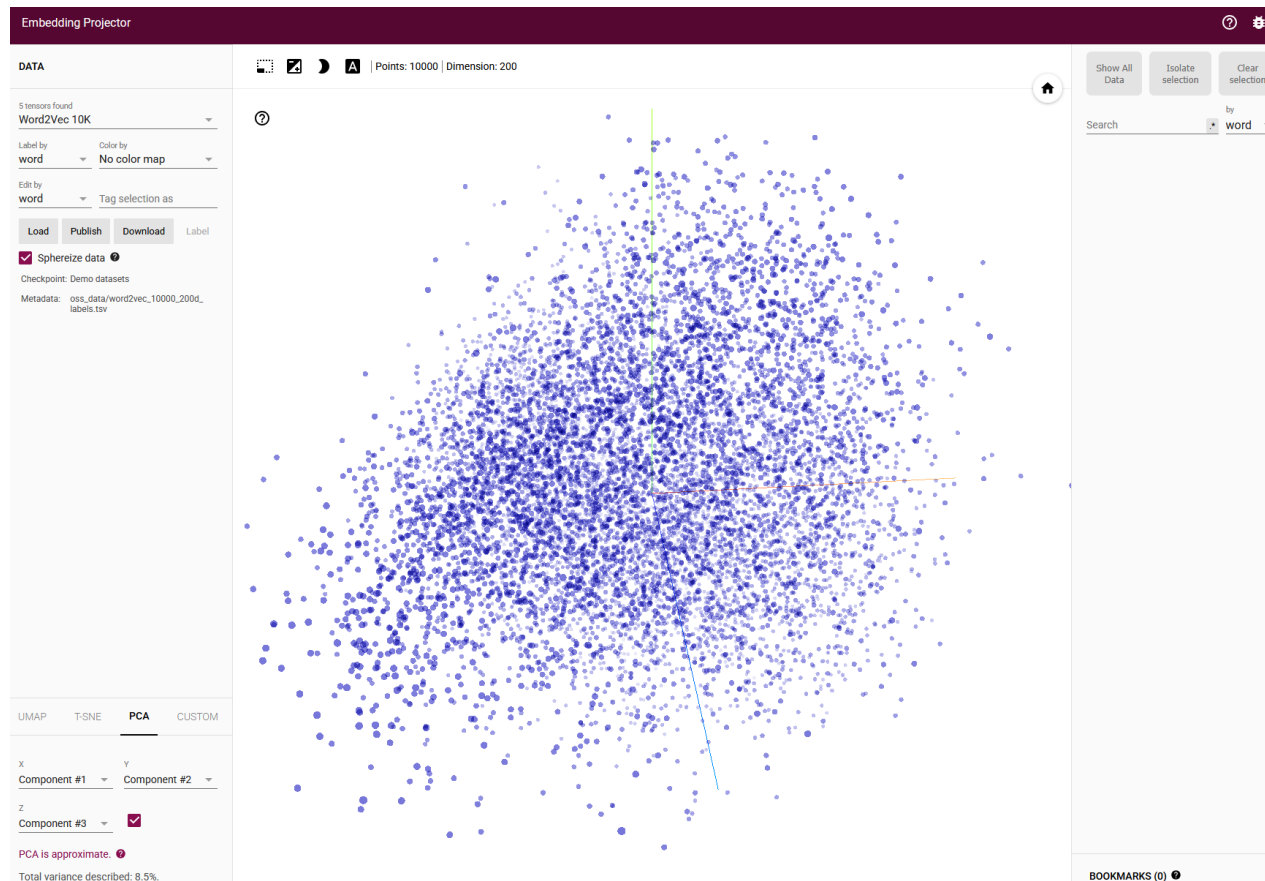
La clave es que se pueden entrenar con aprendizaje no supervisado: no hacen falta etiquetas, solo textos.

Hay varios métodos:

- CBOW: Predecir la palabra central dada su contexto (palabras vecinas). Se suman (o promedian) los vectores de contexto y se usa ese vector para estimar la probabilidad de la palabra objetivo.
- *Skip-Gram*: lo contrario – contexto a partir de la palabra central.



Word2Vec es el modelo de *embeddings* más popular. Ampliamente usado desde hace más de 10 años. Modelos en varios idiomas, diferentes tamaños a escoger, etc.



¡Vamos a visualizar embeddings!

<https://projector.tensorflow.org/>



6. Casos de uso





EBIS Business
Techschool

www.ebiseducation.com

